

Site AI & How It Connects

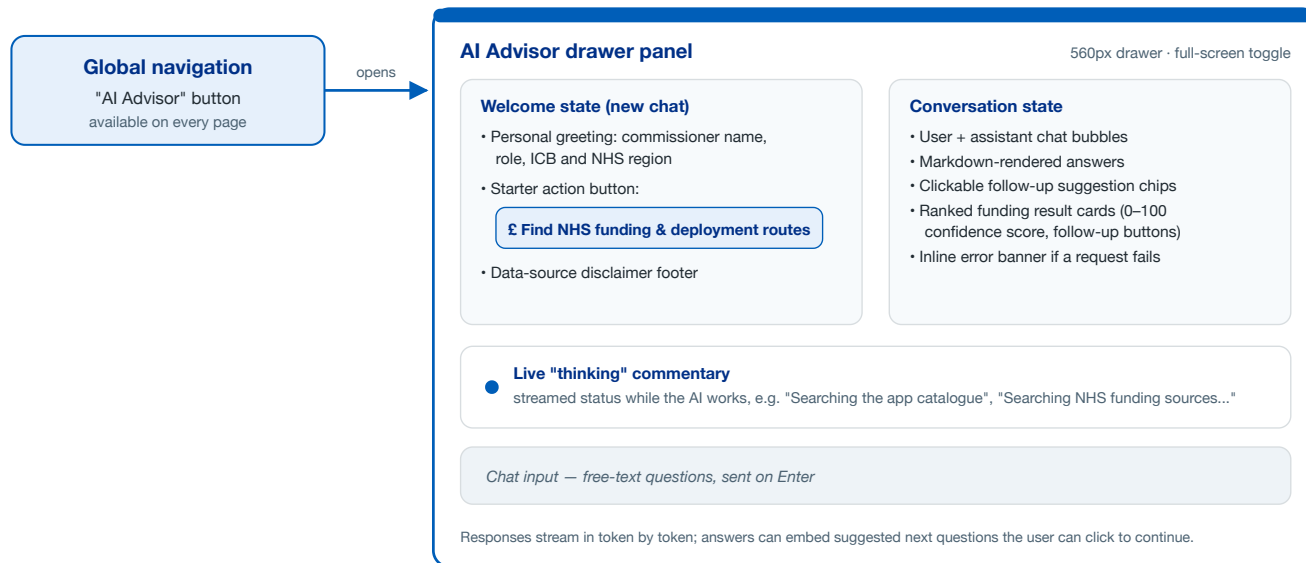
A visual guide to the AI functionality built into the NHS HealthStore site: what the user sees, the services behind it, and how every part connects — from the browser panel through the API layer to the model, the catalogue data, and back.

1	What the AI is — the AI Advisor	The single AI surface: a chat drawer for commissioners
2	System architecture	Browser → API → OpenAI → data, end to end
3	Chat request lifecycle	Gates, personalisation, the tool loop, streaming
4	DTx Funding Finder pipeline	LLM candidates, deterministic scoring, ranked cards
5	Advisor tools & data sources	The 9 function tools plus live web search
6	Connection reference	Streaming events, key files, active configuration

Scope

This document covers only the AI functionality that is currently wired into the running application. Legacy, hidden, or disabled modules and any code paths that are not reachable from the live user interface are deliberately excluded.

The site has **one AI feature: the NHS HealthStore AI Advisor** — a conversational assistant for NHS commissioners. It opens as a slide-in drawer (with an optional full-screen view) from the "AI Advisor" control in the global navigation, and stays available on every page of the site. It is powered by OpenAI's `gpt-5.4` and answers with live data from the HealthStore catalogue.



What the Advisor can do for the user

Explore the catalogue

Search, explain, and compare digital health apps — evidence, pricing, maturity, DTAC status — answered from live HealthStore catalogue data.

Find NHS funding

Run the DTx Funding Finder for a region and condition (or app) and get ranked, scored funding opportunity cards directly in the chat.

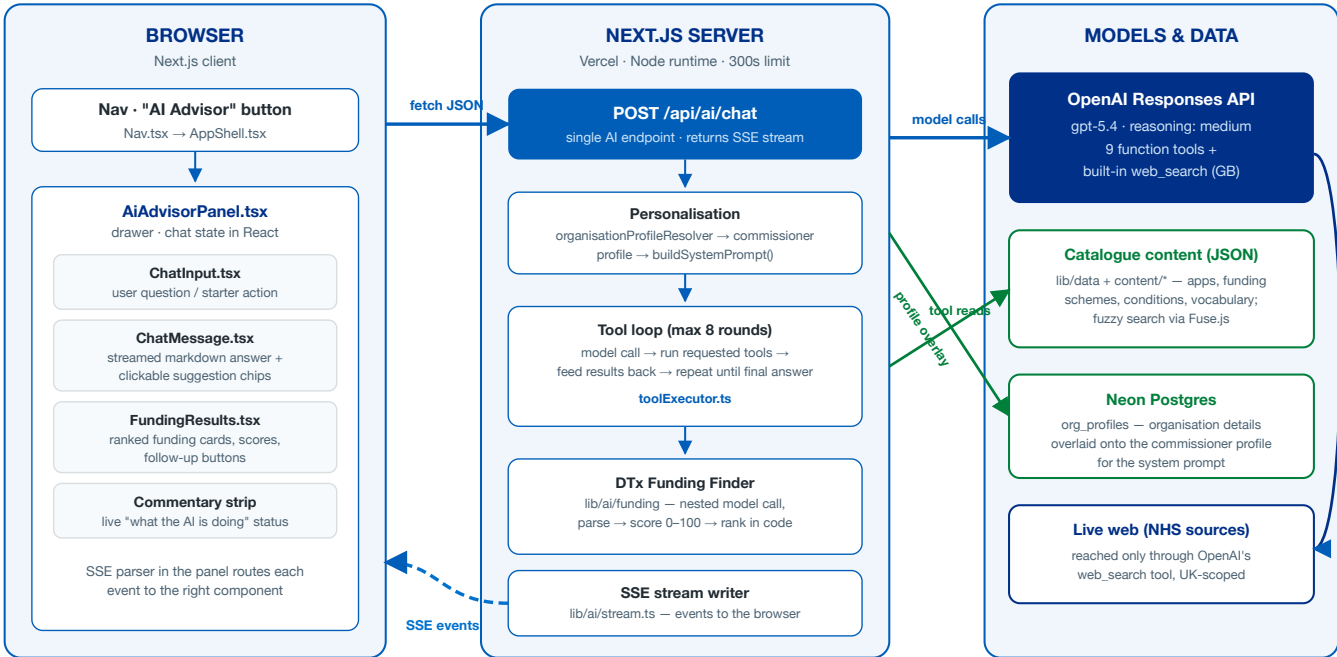
Model costs & business cases

Pull pricing, ROI notes, tariff and procurement information for any app to support commissioning decisions.

Search the live web

Supplement catalogue answers with current NHS sources via built-in web search (UK-scoped).

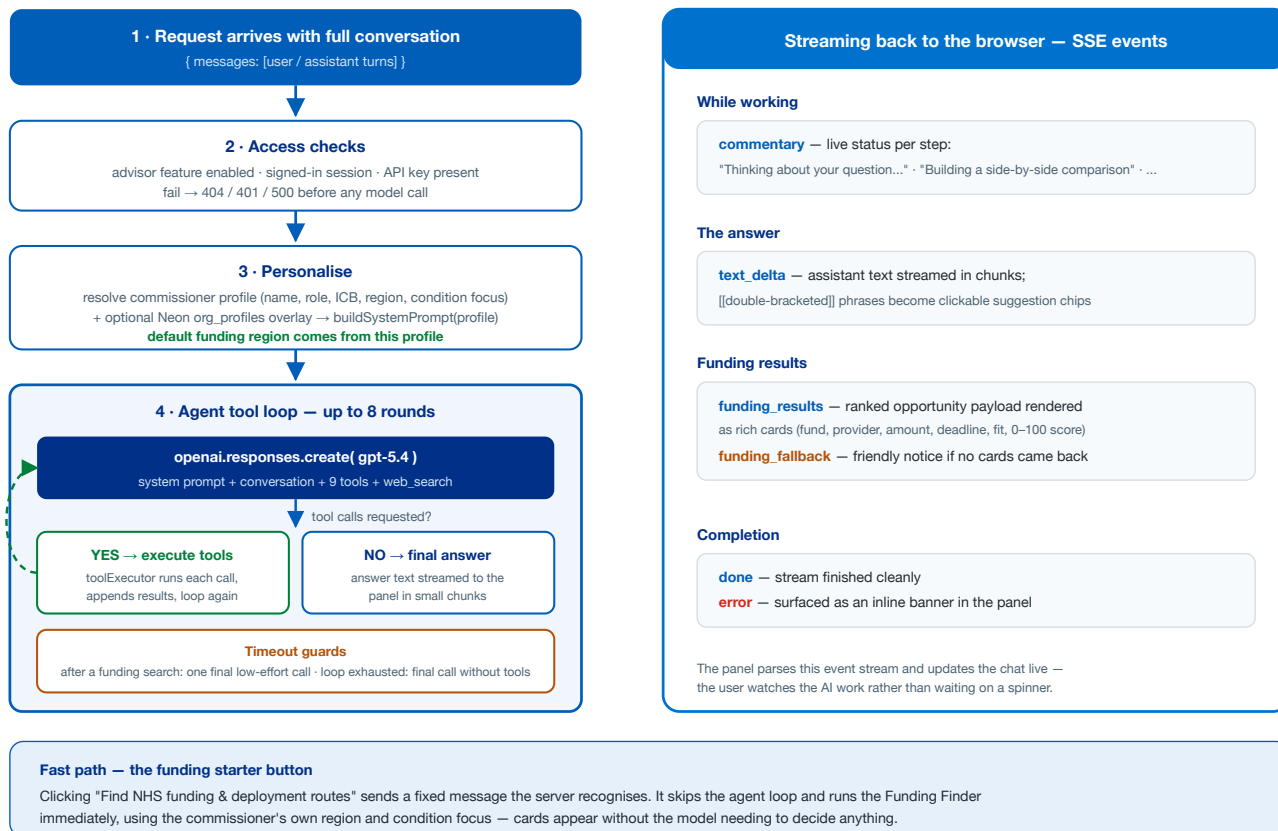
Everything runs through a single streaming endpoint: the panel posts the conversation to `POST /api/ai/chat`, the server orchestrates the model and its tools, and results stream back over Server-Sent Events (SSE). No chat history is stored — the conversation lives only in the browser.



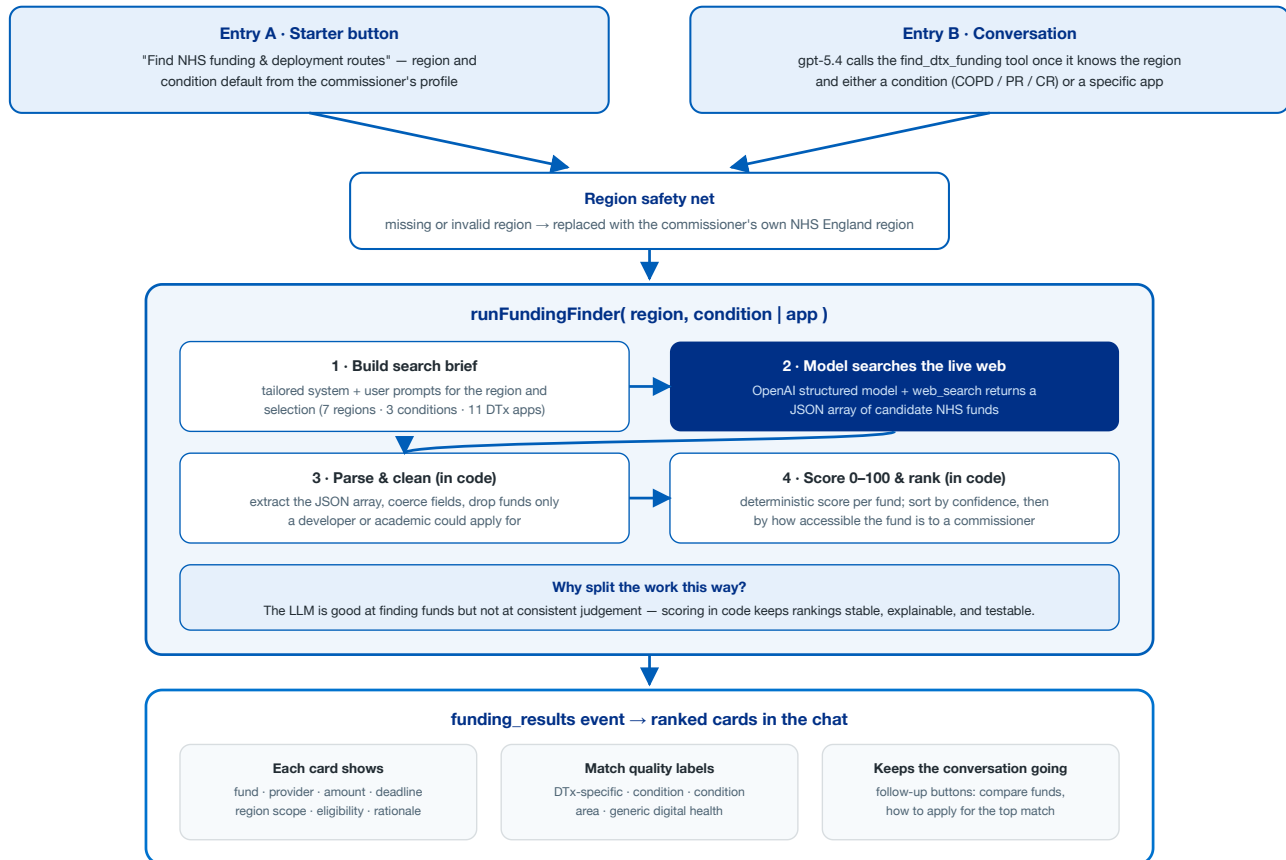
How a message flows

1. The panel sends the whole conversation to the API. 2. The server personalises a system prompt, then lets gpt-5.4 call tools that read catalogue JSON, run the funding finder, or search the web. 3. Text, status commentary, and funding cards stream back to the browser as SSE events while the AI works.

Each message triggers one server request that runs an agent loop: the model may answer directly, or ask for tools; the server executes them and feeds results back, up to **8 rounds**, while streaming progress to the user throughout.



The Funding Finder is the Advisor's most involved capability. The model is only trusted to *find candidates* — everything that ranks them is deterministic code, so scores are consistent and repeatable.



The model never touches data directly. It requests one of these tools; the server executes it and returns the result. Nine function tools read HealthStore data, and OpenAI's built-in web search covers anything live.

Tool	What it does for the user	Data source	Shown as
<code>search_apps</code>	Searches the catalogue by keyword and/or clinical condition; returns a summary list of matching apps.	Catalogue JSON + Fuse.js fuzzy search	Answer text
<code>get_app_detail</code>	Full product dossier for one app: clinical info, evidence, NICE guidance, maturity, DTAC, supplier, service wrap.	Catalogue JSON	Answer text
<code>get_app_financials</code>	Pricing model, indicative cost, tariff, ROI notes, procurement routes — for costs and business cases.	Catalogue JSON	Answer text
<code>compare_apps</code>	Side-by-side comparison of 2–3 apps on commissioning dimensions (price, evidence, maturity, DTAC...).	Catalogue JSON	Answer text
<code>list_funding</code>	Lists curated funding schemes (open, upcoming, periodic), optionally filtered by condition.	Curated funding JSON	Answer text
<code>get_funding_detail</code>	Full detail on one scheme: sponsor, value, eligibility, dates, application notes.	Curated funding JSON	Answer text
<code>get_condition_overview</code>	Landscape view of a clinical programme area and the apps available in it.	Catalogue JSON	Answer text
<code>get_enums</code>	Standard HealthStore vocabulary: pricing models, maturity levels, evidence tiers, DTAC statuses...	Shared enums JSON	Answer text
<code>find_dtx_funding</code>	Runs the DTx Funding Finder (page 5): live search, scored 0–100, filtered to funds a commissioner can access.	Live web via nested model call	Funding cards
<code>web_search</code>	OpenAI built-in live web search, scoped to the UK, for current NHS information beyond the catalogue.	Live web (via OpenAI)	Answer text

Conditions & coverage the tools understand

Catalogue tools — 7 clinical areas

COPD · cardiac rehab · pulmonary rehab · insomnia · weight management · MSK · eating disorders

Funding Finder — focused scope

7 NHS England regions · 3 conditions (COPD, pulmonary rehab, cardiac rehab) · 11 named DTx apps

Guardrails users benefit from

Grounded answers

Catalogue questions are answered from tool results, not model memory; the panel reminds users to verify with suppliers before procurement.

No invented funding

The funding tool's own instructions forbid inventing funds — only parsed, scored results from the search are shown, and a fallback notice appears if none return.

Key modules and how they connect

Module	Layer	Role in the chain
<code>components/Nav.tsx</code> / <code>AppShell.tsx</code>	Browser	"AI Advisor" nav control; opens/closes the drawer panel
<code>components/ai/AiAdvisorPanel.tsx</code>	Browser	Chat UI, conversation state, SSE parsing and event routing
<code>components/ai/ChatMessage.tsx</code>	Browser	Renders answers; turns <code>[[phrases]]</code> into suggestion chips
<code>components/ai/FundingResults.tsx</code>	Browser	Ranked funding cards with scores and follow-up actions
<code>app/api/ai/chat/route.ts</code>	Server	The single AI endpoint: gates, agent loop, streaming
<code>lib/ai/config.ts</code>	Server	Shared OpenAI client; model and reasoning settings
<code>lib/ai/systemPrompt.ts</code> + <code>organisationProfileResolver.ts</code>	Server	Personalised system prompt from commissioner profile (+ Neon overlay)
<code>lib/ai/tools.ts</code> + <code>toolExecutor.ts</code>	Server	Tool definitions given to the model; server-side execution against catalogue data
<code>lib/ai/funding/*</code>	Server	Funding Finder: prompt → model+web search → parse → score → rank
<code>lib/ai/stream.ts</code>	Server	SSE writer — the wire between server progress and the panel

Streaming events (the browser-server contract)

Event	Meaning	Panel behaviour
<code>commentary</code>	What the AI is doing right now	Status strip above the input
<code>text_delta</code>	Next chunk of the assistant's answer	Appends to the answer bubble
<code>funding_results</code>	Ranked funding payload for a search	Renders funding cards
<code>funding_fallback</code>	Search ran but returned no cards	Shows a gentle notice
<code>error</code>	Something went wrong server-side	Inline error banner
<code>done</code>	Response complete	Re-enables the input

Active configuration

Setting	Value	Effect
Chat model	<code>gpt-5.4</code>	All advisor conversation turns
Reasoning effort	<code>medium</code> (final wrap-up: <code>low</code>)	Depth vs latency balance per call
Max tool rounds	<code>8</code>	Caps agent loops per message
Function timeout	<code>300s</code> (Node runtime)	Headroom for web-search-backed funding runs
Web search locale	<code>GB</code>	UK-scoped live results
Access	Feature-flagged + signed-in session	API refuses requests before any model call otherwise
Persistence	None	Chats are ephemeral; nothing is written to the database by the AI

One surface, one endpoint

Every AI capability on the site — catalogue Q&A, comparisons, cost modelling, funding search, web lookups — flows through the AI Advisor panel and its single streaming endpoint. There are no other AI features in the running application.